Boundary Case — Vendor Unavailable: How a CKS Deployment Continues Operation When a Vendor Providing Architectural Function Becomes Unavailable

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 7, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one boundary case under which a CKS deployment must continue operating — vendor unavailability — by stating the architectural treatment the source paper's commitments imply when a vendor providing substrate hosting, authoritative content storage, LLM consultation, or another architectural function becomes operationally unavailable, naming the canonical anti-pattern responses, and observing the limits of the formalization.

Abstract

The CKS pattern's tool-agnosticism commitment names three minimal requirements for any substrate host (persistent structured state, human read/write access, LLM access to substrate content) and treats the substrate's host interface as a specification rather than any particular vendor's implementation. The commitment is normally exercised as design freedom — the architect chooses among hosts that satisfy the requirements. This note formalizes the boundary case in which exercise is forced rather than chosen: a vendor providing one of the architectural functions becomes operationally unavailable — through outage, business failure, regulatory restriction, geographic unavailability, sanctions, or contractual dispute — and the deployment must continue operating, degrade gracefully, or migrate to an alternate vendor on a timeline the deployment did not pick. The note states the architectural treatment, names six anti-pattern responses that violate the architecture (vendor lock-in revealed during outage, authority lost during migration, provenance lost during migration, compliance frameworks locked to a specific vendor, manual recreation of state, data-export-only with no rules), distinguishes the boundary from related cases (composition partner failure, LLM consultation failure within cells, network partition), and provides an operational test for whether a deployment instantiates the boundary's architectural treatment.

1. Why the vendor unavailable boundary needs to be formalized as standalone

Tool-agnosticism is normally formalized as a design-time property: the architect chooses among environments that satisfy three minimal requirements (§7.1 of the source paper), and the substrate's host interface is a specification rather than any particular vendor's implementation. A separate

operational-test formalization establishes proactive verification — the practice of confirming, before any forced unavailability, that a deployment's substrate can in fact migrate between qualifying hosts. Both treat tool-agnosticism in a regime where migration is available as an option but not required.

Vendor unavailability is the regime in which migration ceases to be optional. The vendor providing one of the architectural functions — substrate hosting, authoritative content storage, the LLM operating as substrate mediator, a composition partner the deployment depends on — becomes operationally unavailable, and the deployment must continue, degrade, or migrate without that vendor in the loop. The causes are operationally varied: outages of any duration; business failure or acquisition that retires the service; regulatory restrictions imposed in particular jurisdictions; geographic unavailability following sanctions or trade restrictions; contractual disputes that suspend access. What the causes share is that the deployment did not choose the timing, the duration, or in many cases the destination.

Formalizing the boundary as standalone matters for two reasons. First, vendor unavailability stresses tool-agnosticism in operationally distinct conditions from proactive verification: it tests whether the substrate is genuinely host-agnostic when the architect cannot tune the migration plan in advance and may have to settle for less than optimal conditions on the receiving end. Second, vendor outages are the moment at which architectural violations not visible in normal operation become visible all at once; if the deployment cannot continue without a particular vendor, the architecture had a vendor-specific dependency the design pretended not to have, and the outage is the first time anyone has cause to notice. The "AI vendor lock-in" framing widely named in 2024–2026 makes the topic operationally current.

2. The boundary case scenario and what makes it non-obvious

A vendor providing one of the architectural functions becomes operationally unavailable, the unavailability is outside operators' control, and the deployment continues to have coordination work to do. The non-obvious questions concern scope, severity, and choice.

Scope. Different vendors provide different architectural functions, and unavailability of each has different consequences. *Substrate hosting unavailable*: the host environment carrying substrate content becomes unreachable, and coordination operations cannot proceed at all until the substrate is migrated to an alternate host or the original is restored — the most operationally severe case. *Authoritative content provider unavailable*: a vendor maintaining a category of substrate content becomes unavailable; the substrate may continue, but the affected content must be migrated, reconstructed, or formally marked as currently inaccessible substrate state. *LLM consultation vendor unavailable*: cells whose execution depends on LLM consultation cannot proceed, while substrate operations that do not require consultation continue — the mediator role is suspended for affected cells, the substrate is otherwise unaffected. *Composition partner vendor unavailable*: the substrate continues; the composition surface that the unavailable partner occupied must be reconfigured. The functions are operationally distinct, and a deployment may lose access to one without losing access to the others.

Severity. A short outage may be best handled by waiting; a long outage requires migration; a permanent unavailability — vendor business failure, sanctions of indefinite duration, regulatory withdrawal — requires migration on a determinable timeline. The architecture supports all three responses; the choice among them is operational.

Choice. Two architectural responses are available, and a deployment may use both at different scopes within the same incident: graceful degradation (operations queued or refused with clear error states; the substrate continues for unaffected operations) and migration (authoritative content moved to an alternate vendor; the deployment continues fully). A short outage of an LLM consultation vendor is well-served by graceful degradation; a permanent loss of a substrate hosting vendor requires migration; an authoritative-content-provider failure may need both, in sequence. None of these responses is uniformly correct, and the architectural treatment is what makes them all available without committing to any one of them at design time.

3. Which architectural commitments are stressed

Vendor unavailability stresses several CKS commitments at once. *Tool-agnosticism* is stressed in operationally forced conditions rather than as design freedom: the substrate must be hostable on whatever alternate environment is available now, even if that environment was not the architect's first choice and lacks vendor-specific features the operators valued. *Substrate-as-source-of-truth* (§11.3) is stressed because authoritative content must move during migration; if any category is encoded in a form only the unavailable vendor can interpret, that category does not migrate, and the substrate fails on the source-of-truth commitment for that category at the moment the vendor goes away. *Linear-cost scaling* (§6) is stressed because the destination host may have materially different cost characteristics; a deployment that migrates but lands on a host where storage is quadratic where the source was linear retains tool-agnosticism but loses linear-cost. Both commitments are required; either alone is insufficient. *Composition requirements* are stressed where the migration affects partners, or where a partner is itself affected by the same unavailability — the composition surface is rebuilt under operational stress, and the requirement that composition preserve governance, retraceability, and determinism applies under that stress. *Path retraceability* is stressed across the migration boundary: provenance metadata every piece of substrate content carries — writer, timestamp, rule reference, antecedents, rationale, conflict relationships — must survive the move; if any field is lost, the retraceable path runs through information the migrated substrate no longer carries, and retraceability fails for content older than the migration. *The human-governed commitment* is stressed because the three rights — to inspect, to modify, to override substrate content and orchestration rules at any time — must remain available on the destination as a property of its design, not as a feature of the source vendor's environment that may or may not be reproduced.

The boundary is the moment several CKS commitments are simultaneously tested in operational conditions the deployment did not choose.

4. The architectural treatment

The treatment has five components, applied as the situation warrants.

(a) Continue for unaffected operations. Whatever substrate operations do not require the unavailable vendor proceed normally. If the LLM consultation vendor is unavailable, substrate reads, human writes, and cell executions that do not call the LLM continue. If a composition partner is unavailable, substrate operations that do not require that partner continue. Continuation for unaffected operations is what makes vendor unavailability survivable in the first place — the substrate is not all-or-nothing in any direction.

(b) Graceful degradation for affected operations. Operations that require the unavailable vendor are either queued for retry (where duration is short and queueing is acceptable) or refused with clear error state (where duration is unknown and the operation must report failure to its caller). The error state is itself substrate content where appropriate — a cell that cannot consult an unavailable LLM may write a substrate record indicating the consultation was attempted and failed, with the affected substrate object marked as having an unresolved dependency. Degradation is not silent; what degraded, when, and why is recorded as substrate state.

(c) Migration for forced cases. Where unavailability exceeds the duration the deployment can absorb, or where conditions make the unavailability permanent, the substrate migrates to an alternate vendor. The migration is the architectural exercise of tool-agnosticism: the substrate's host interface is the three minimal requirements, and any environment satisfying them is a candidate destination. The migration moves authoritative content across all source-of-truth categories — decisions, decision provenance, active contradictions, conflict resolutions, orchestration rules, authority assignments. It moves provenance across all six provenance fields — writer, timestamp, rule reference, antecedents, rationale, conflict relationships. It verifies that linear-cost characteristics hold on the destination and that the inspect, modify, and override rights remain exercisable on the destination by the same humans who exercised them on the source.

(d) Composition reconfiguration. Where the migration affects composition partners — the partner's connection point now references the new host, or the partner is itself affected by the same unavailability — the composition surface is reconfigured per the composition requirements. Governance preservation, retraceability preservation, and determinism preservation across composition each apply to the new connection point as they did to the old.

(e) The migration is recorded as a substrate event. The migration itself is a change to substrate state and is recorded as substrate content with full provenance: the writer, the timestamp, the rule under which the migration was authorized, the antecedent state, and the rationale (the unavailability that prompted the migration). After the migration, a reader reading substrate content alone can reconstruct the fact of the migration and the conditions under which it occurred — the migration is not an external event the substrate forgets.

The five components do not specify which vendor a deployment migrates to, which migration tooling is used, or what business-continuity processes wrap the migration; those are deployment decisions outside the architecture's scope. What the architecture commits to is that the components above are available as architectural responses under any vendor unavailability the deployment encounters.

5. Anti-pattern treatments that would violate the architecture

Six anti-pattern responses are nameable, each violating the architecture in a specific way.

Vendor lock-in revealed during outage. The deployment cannot migrate because authoritative state is held in a form only the unavailable vendor can interpret, only its API exposes, or only its runtime makes operational. The lock-in was not visible while the vendor was available; the outage reveals it. Three canonical violations from the anti-pattern series surface here together: substrate substitute (a vendor's product treated as a CKS substrate without satisfying the three minimal requirements), pure context-window memory as substrate (coordination state held in the LLM's context across turns rather than in persistent structured form), and black-box agent memory as substrate (memory held inside an

agent framework only the framework can read). Each is invisible in normal operation and visible the moment the relevant vendor is unavailable.

Authority lost during migration. The authoritative content is migrated, but in a form the destination cannot represent — relationships flattened, rule references stripped, conflict edges collapsed into errors. The substrate appears to migrate; what actually migrated is data without its authority structure. The deployment now has substrate content on the destination it cannot answer coordination questions from with the same authority as before.

Provenance lost during migration. The data migrated; the provenance did not. Writer attribution is replaced with "imported from prior system." Timestamps are reset to the migration date. Antecedent references are dropped because the destination does not support them. The retraceable path runs back to the migration event and stops; path retraceability fails for everything older than the migration.

Compliance frameworks locked to a specific vendor. The deployment's certifications, audit attestations, or regulatory acceptances were issued against the source vendor's infrastructure rather than against the substrate as an architectural object. Migration invalidates the certifications, and the deployment must either operate uncertified pending re-attestation or remain on the unavailable vendor pending the original certifications' expiry. The architecture did not commit to vendor-locked compliance; the deployment's compliance posture did.

Manual recreation of state. Authoritative content is not migrated architecturally; it is recreated by humans reading the source content through whatever access remains and re-entering it on the destination. Recreation discards provenance (writers and timestamps cannot be recovered), discards conflict structure (active contradictions are silently resolved by whoever does the re-entry), and is bounded by the recreators' attention rather than by the substrate's actual content. What ends up on the destination is a different substrate that resembles the source; it is not the same substrate hosted elsewhere.

Data-export-only, no rules. The vendor offers a data export, and the deployment migrates the data. Orchestration rules, authority assignments, and conflict structure — also substrate content per §11.3 — are vendor-specific and do not export. The destination has substrate content but not the rules under which cells operate over it; cells re-implemented on the destination do so under whatever rules a human writes from memory, with no architectural guarantee of equivalence. This is the most subtle violation, because the migration looks complete: the data is all there. The rules are gone.

A deployment that responds to vendor unavailability with any of the six is exercising a substitute for the architectural treatment, not the treatment itself. Naming the violations is what makes downstream remediation possible.

6. Operational implications

Three implications are worth stating explicitly. *Different functions imply different responses.* A deployment that loses LLM consultation may operate degraded for hours without urgency; one that loses substrate hosting must migrate or pause within whatever time horizon the coordination work allows; one that loses an authoritative-content provider must decide function by function whether to migrate, reconstruct, or mark as inaccessible. The architectural treatment is the same — continue for unaffected operations, degrade or migrate for affected ones — but the operational rhythm differs across functions, and a deployment's runbook should differentiate accordingly. *Orchestration rules may*

require revision. Where a rule references vendor-specific operational details, the rule's reference must be updated to the destination's equivalent. The semantic content of the rule does not change; the operational reference inside it does. The rule revision is itself a substrate event with its own provenance. *The migration itself surfaces the architecture's actual posture.* Vendor unavailability is the moment at which a deployment learns, in operationally consequential terms, what it actually committed to. A deployment that migrates cleanly with provenance and authority intact had a vendor-independent architecture; a deployment that cannot migrate had vendor-specific dependencies the design did not foreground. The boundary is diagnostic as well as operational.

7. Limits of the architectural treatment

The treatment is scoped to vendor-level unavailability. Three adjacent boundary cases require their own treatments and are not addressed here. *Composition partner failure not at the vendor level* — a Pattern A consultation that returns errors due to schema drift, a derived view that goes stale because its refresh logic is broken, a separate concern whose interface contract was violated — is a partner failure rather than a vendor unavailability, and a separate boundary case is the appropriate site for it. *LLM consultation failure within cells* — an LLM that is reachable but is producing failed or invalid responses to cell-level consultations — is a cell-execution failure, not a vendor outage; the cell-internal behavior under such failures is the proper subject of a different boundary case. *Network partition where the vendor is reachable but slow* is not unavailability in this note's sense; latency thresholds at which graceful degradation should trigger, and the architectural treatment of long-tail latency, are operationally distinct from binary unavailability.

Proactive verification (the tool-agnosticism-migration test) and operationally forced migration are related but architecturally distinct: proactive verification establishes that the deployment *can* migrate; the boundary case here formalizes what happens when it *must*. The two are complementary; this note does not subsume the other, and a deployment that has not exercised the proactive verification is not in a stronger position simply because it has read the boundary case formalization.

The treatment is also silent on which alternate vendor a deployment should choose, which migration tools are appropriate, and how the deployment's runbook should sequence the response. These are deployment decisions; the architecture's contribution is the specification of what the boundary's architectural response must preserve.

8. Operational test

A deployment instantiates the architectural treatment of the vendor-unavailable boundary if and only if all of the following are true at the moment vendor unavailability begins:

1. Substrate operations not requiring the unavailable vendor continue without architectural change.
2. Operations requiring the unavailable vendor either degrade gracefully (queued, or refused with clear, substrate-recorded error state) or are addressed by migration to an alternate vendor that satisfies the three minimal requirements of the substrate-host interface.

3. Migration, when undertaken, preserves all source-of-truth categories of authoritative content and all six provenance fields without loss; the inspect, modify, and override rights remain exercisable on the destination by the same humans who exercised them on the source; and the linear-cost characteristics hold on the destination as on the source.
4. The migration itself is recorded as substrate content with full provenance — writer, timestamp, rule reference, antecedent state, rationale.
5. Composition partners affected by the migration are reconfigured under the same composition requirements (governance preservation, retraceability preservation, determinism preservation) that applied at design time.

A deployment that fails any of (1)–(5) may continue to function in some operational sense, but is not exercising the boundary's architectural treatment as the architecture defines it.

9. Why naming the boundary as standalone matters

Vendor unavailability is one of the moments at which a deployment's architectural posture becomes operationally consequential, and one of the moments at which architectural violations not visible in normal operation become visible in a single event. Treating the boundary as standalone makes three things possible. A deployment can ask, before any vendor is unavailable, whether its response would satisfy the operational test in §8, and where the answer is no, the gap names a specific architectural change to make. The six anti-patterns in §5 are common enough in practice that a deployment encountering vendor unavailability for the first time may default into one of them and call the result a successful migration; the standalone formalization is what makes those defaults recognizable as architectural violations rather than as reasonable operational choices forced by the situation. Subsequent Phase A6 notes — LLM consultation timeout, composition partner failure, network partition, and others — refer back to the vocabulary developed here for what migration preserves and what graceful degradation looks like; each is operationally distinct from vendor unavailability, but the architectural commitments that survive each of them are the same set, and naming the set here makes the subsequent notes shorter and more precise.

The boundary is one in a series; the series stress-tests the foundational architectural commitments under conditions deployments did not choose. Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should treat vendor unavailability — and the larger class of operationally forced architectural exercise — as the moment at which the architecture's claims are paid in full or paid in part. Naming this boundary as standalone is what makes the difference visible.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Boundary Case — Vendor Unavailable: How a CKS Deployment Continues Operation When a Vendor Providing Architectural Function Becomes Unavailable.* May 7, 2026. ORCID:

0009-0004-8065-3235.